

Audio on a static site: build-time TTS and the script it reads from

Emil Lindfors | August 15, 2026

My partner is dyslexic. It had not occurred to me until recently that a 2,500-word post about OTLP temporality is, for a good number of people, work rather than reading.

So the site can now generate an audio version of a post at build time. This one has it, at the top of the page. The rest of the back catalogue is switched off for now: I ran the pipeline over all eight posts once to see what it came to, got 97 minutes and 45 MB, and haven't decided whether I want that in git. This is how it works and what went wrong on the way.

What the research says

The evidence for text-to-speech and dyslexia is better than I expected. A 2018 meta-analysis in the *Journal of Learning Disabilities* found read-aloud presentation improves comprehension for readers with reading disabilities, and the mechanism is straightforward: decoding costs effort, and the effort comes out of the same budget as comprehension.

The finding that changed my design was that the strongest effect is *bimodal* — audio plus synchronised highlighting of the word being spoken beats audio alone and beats silent reading. That's stage four for me and it isn't built yet, but it's the reason the pipeline emits an intermediate artifact instead of going straight from markdown to MP3.

One more thing: the older robotic voices raise cognitive load again, which cancels part of the benefit. `speechSynthesis` in the browser is free and I'm not using it.

Why build time

Three options: synthesise in the browser, synthesise on request in a Cloudflare Function, or synthesise at build time and commit the result.

The browser option is free and immediate, and it sounds like a 2009 satnav. The Function option puts a GPU in the request path for a page that is otherwise static files on a CDN. So: build time, committed, the same way the post PDFs already work. Cloudflare Pages runs its own `zola build` on push, so anything not in the repo doesn't survive the trip.

Storage is fine. Mono at 64 kbps gives 7.2 MB for the longest post, which runs 15:46, and 45 MB for the whole archive, against Cloudflare’s limit of 25 MiB per file and 20,000 files. Git will carry that. If it stops being fine, R2 is the documented escape hatch.

The part that took the longest

Roughly a third of this blog by volume is code fences, tables, ASCII diagrams and a rendered reference list. None of that is listenable. Handing raw markdown to a TTS model gets you a voice reading `#[derive(Debug)]` out loud, and then reading a DOI.

So there’s a step before synthesis. `site-tools speech` derives a spoken script and writes it to `static/speech/<slug>.txt`, committed and diffable. When the audio sounds wrong, that file is where I look, and it’s cheaper to read than to listen to.

In the post	In the script
Fenced code block	<code>Rust code block</code> , 14 lines. See the article.
Box-drawing diagram	Diagram, 15 lines. See the article.
Table	Table, 7 rows. See the article.
<code>![alt](src)</code>	Figure: alt. — dropped if alt is empty
<code>\$\$...\$\$</code>	Equation.
<code>## References</code> and everything after	dropped
<code>[text](url)</code>	text
<code>et al.</code>	and others

Structure survives as whitespace. One blank line between blocks is a paragraph gap, two is a section break, and the synthesis step turns those into 0.45 and 0.9 seconds of silence. Keeping the structure in the readable file meant I didn’t need a parallel manifest describing it.

The transformation has to be fence-aware, which I knew going in because two of these posts quote Tera inside code fences. A stripper that didn’t track fences would delete the examples the posts exist to show.

What it got wrong first

I generated all eight scripts, read them, and found the extraction had mangled things in ways I would otherwise have discovered by listening to 97 minutes of audio.

`client_max_body_size` came out as `clientmaxbodysize`. I was stripping `_` as an emphasis marker without checking whether it was inside a word. Same for `codex_turn_token_usage` and `genai.usage.total_cost`. The fix is to treat inline code differently from prose: inside backticks `_` becomes a space, outside it’s emphasis and gets dropped.

`~/fraktal/config.toml` came out as `/fraktal/config.toml`, because I was stripping `~` as a strikethrough marker. Only `~` is strikethrough. A lone tilde is a home directory.

Bullet lists were merging into one block, so three list items became one 40-second sentence with no breath in it. Each item is its own block now.

And in the two posts about Zola templates, inline Tera like `{{<citation key="smith2024" num="1" />}}` was going through to the model verbatim. There is no good pronunciation for that. It now becomes “this tag”, which reads acceptably in a sentence — “in a post, this tag renders as a clickable 1” — and fires six times in the worst post. Posts that are mostly syntax can opt out entirely with `extra.skip_audio`, mirroring the `extra.skip_citations` opt-out I already had. It is doing broader duty at the moment, since every post except this one has it set.

None of these were crashes. They were plausible output that happened to be wrong, which is the category of bug I keep writing about.

Picking a model

I’m running Fish Audio S2.1 Pro. Two reasons: it’s good, and I can move it onto my own GPU later without changing anything but a URL. The pipeline knows a base URL and a model string, and both come from `.env`.

Cost turned out not to be a factor at all. The whole archive is 84,327 characters of speech script. At the going rates that’s:

Service	Rate	Whole archive
Google Cloud Standard	\$4/M	\$0.34
OpenAI <code>tts-1</code>	\$15/M	\$1.26
Cloudflare Aura-1	\$15/M	\$1.26
ElevenLabs Flash	\$50/M	\$4.22

I paid nothing, because `s2.1-pro-free` is free through the end of August 2026. But even the expensive column is one coffee for the entire back catalogue, so the decision came down to voice quality and whether I could self-host it later.

There are two backend shapes in the code, chosen by an enum: Fish’s `POST /v1/tts`, and the OpenAI `POST /v1/audio/speech` shape, which also covers Kokoro-FastAPI and the OpenAI-compatible Fish wrappers. A closed set of two, so static dispatch and no trait object. HTTP goes through `curl`, the way the newsletter sender already does — `site-tools` depends on `toml` and one git crate, and a build-time task doesn’t justify an async runtime.

Blocks, not posts

Each block goes to the API on its own rather than posting the whole post in one call, for three reasons that all turned out to matter:

- S2.1 Pro is autoregressive, and long generations drift.
- A block that comes back wrong is retried alone.
- Editing one paragraph costs one API call instead of seventeen minutes of GPU time.

The blocks come back as WAV and get joined with ffmpeg's concat demuxer, then a single encode to MP3. Stitching MP3s instead would put a frame-boundary gap at every block and add a second generation of encoding loss.

The silence between blocks is cut from a real block rather than generated:

```
ffmpeg -i block.wav -af volume=0 -t 0.45 gap-paragraph.wav
```

That guarantees the sample rate and channel count match the blocks exactly, which the concat demuxer requires. I could have probed the model output and constructed matching silence, but this is one command and can't drift.

Then one pass over the whole thing:

```
ffmpeg -f concat -safe 0 -i concat.txt \  
-af loudnorm=I=-16:TP=-1.5:LRA=11 \  
-ac 1 -codec:a libmp3lame -b:a 64k out.mp3
```

`loudnorm` at -16 LUFS is the podcast convention. Without it, volume drifts between posts synthesised weeks apart. Measured after the fact, the first post came out at -16.6 LUFS integrated, which is close enough that I stopped looking at it.

Two bugs in this part were mine and both would have shipped silently. An HTTP 200 is not proof of audio: an API can answer 200 with a JSON error body, and I was caching that as a valid block and concatenating it. It checks for a RIFF header now. And when a block failed all its retries, curl had already written the error body to the output path, where the next run would find it and treat it as cached audio.

The gate

Every block is hashed, and the whole script is hashed. If the script hash matches what's recorded in the sidecar and the MP3 exists, the command makes no network calls at all.

This is what keeps the GPU out of the critical path. `build.sh` runs the audio step behind a check for ffmpeg and a reachable endpoint, and treats failure as a warning rather than an error, the same posture the citation step already takes. If my GPU box is asleep and I want to fix a typo and deploy, nothing blocks. The committed MP3 ships unchanged.

FNV-1a for the hashing, which is a cache key rather than a signature. The question it answers is "is this block byte-identical to the one I already paid to synthesise".

The player

The player sits above the article body rather than in the sidebar, on the grounds that someone who needs it shouldn't have to hunt for it, and the sidebar is hidden on mobile anyway. It renders only when the sidecar JSON exists, which Zola picks up with `load_data`. That is also how it stays off everywhere else: with no sidecar there is no player and no empty space where one would be.

Native `<audio controls>`, so it's keyboard-reachable and works with the media keys and with JavaScript off. The only scripted part is the speed buttons, and the choice is remembered — someone who listens at 1.5× wants that on the next post too.

One detail I'd have got wrong without testing it in a browser: setting `playbackRate` isn't enough. `load()` resets it to `defaultPlaybackRate`, and with `preload="none"` the audio is fetched long after the restore runs. It survives in Chrome today. It didn't look like it would survive a reload, so both get set.

There's a line under the player saying the narration is synthetic and that code blocks and tables are summarised rather than read out. That seemed like the minimum.

What's missing

Word-level highlighting is the one with the actual research behind it and it isn't built. Fish returns audio bytes and no timings, so the plan is a WhisperX forced-alignment pass over the generated MP3 using the speech script as the known transcript. That's alignment rather than recognition, which is a much easier problem, and it needs the script file that already exists.

A podcast feed is the other obvious thing, and it needs the back catalogue committed first. That would be about two hours of audio, with an Atom feed already sitting next to it.

And I still haven't listened to all of what I generated, which is most of why the other eight aren't switched on. `speech-lexicon.toml` exists for pronunciation overrides and currently has one entry in it — `nginx`, which every TTS model I've tried says as “en-jinx”. “Typst” appears 39 times across the archive and I have no idea yet whether it needs a line in there.

If you use TTS to read things and something on this site comes out wrong, tell me what and I'll fix the extraction. That feedback is hard to get otherwise, since the failure is silent for anyone reading with their eyes.